

26/9/26 DSA JAVA

Second largest distinct number in an array:-

```
→ import java.util Arrays;
public class SecondLargest {
    static Integer bySorting (int[] a) {
        int[] c = Arrays.copyOf(a, a.length);
        // copy made, as we don't want to modify the
        // original array
        Arrays.sort(c);
        for (int i = c.length - 2; i >= 0; i++) {
            if (c[i] != c[c.length - 1])
                return c[i];
        }
        return null;
    }
}
```

// finding 2nd largest element without sorting:-

```
static Integer onePass (int[] a) {
    if (a.length < 2)
        return null;
    long first = Long.MIN_VALUE, second =
    Long.MIN_VALUE;
    for (int x: a) {
        if (x > first) {
            second = first;
            first = x;
        }
        else if (x < first && x > second)
            second = x;
    }
}
```

return second == Long.MIN_VALUE ? null : (int) second;

```
}
public static void main (String[] args) {
    int[][] tests = {
        {12, 35, 1, 10, 34, 13},
        {10, 10, 10},
        {-5, -2, -2},
        {7, 7, 3},
        {42},
        {}
    };
}
```

```
for (int[] t: tests)
    System.out.println ("1-223 sort: 1-5 is
onePass: 1-8-1-0", Arrays.toString(t), bySorting(t),
onePass(t));
}
```

Check for duplicate number in array:-

```
→ import java.util HashSet;
import java.util Random;
import java.util Set;
public class Duplicatecheck {
    // using nested loop:-
    static boolean hasDuplicateNested (int[] a) {
        // compares adjacent elements:-
        for (int i = 0; i < a.length; i++) {
            for (int j = i + 1; j < a.length; j++) {

```

Galaxy M13



```

        if (a[i] == a[j])
            return true;
    }
    return false;
}

static boolean hasDuplicateSet (int[] a) {
    // using HashSet:-
    // HashSet does not allow to add
    // duplicates:-
    Set<Integer> seen = new HashSet<>();
    for (int x : a)
        if (!seen.add(x)) return true;
    return false;
}

public static void main (String[] args) {
    for (int n=10; n<=200; n*=2) {
        int[] a = new Random(7).ints(n, 0,
            Integer.MAX_VALUE).distinct().toArray();
        long t = System.nanoTime();
        boolean e1 = hasDuplicateNested(a);
        long nested = (System.nanoTime() - t) /
            1_000_000; // to convert ns to ms
        t = System.nanoTime();
        boolean e2 = hasDuplicateSet(a);
        long set = (System.nanoTime() - t) / 1_000_000;
        System.out.printf ("n = %.1f, %d nested
            loops: %.1f ms HashSet: %.1f ms (%.1f %.1f) %.1f\n",
            a.length * t, nested, set, e1, e2);
    }
}

```